Reordering vs. Layout: Recovering DRAM Locality for Graph-Based Approximate Nearest Neighbour Search on Processing-in-Memory

senqx
Computer Science — Computer Architecture (HPC)
Master's Thesis Preparatory Report
November 6, 2026

***Abstract*—Processing-in-Memory (PIM) is a promising answer to the data-movement bottleneck that dominates memory-bound workloads. Approximate Nearest Neighbour (ANN) search — the retrieval engine behind recommendation and Retrieval-Augmented Generation — is strongly memory-bound and a natural PIM target. Yet the emerging ANN-on-PIM literature accelerates only the *cluster-based* index family (IVF/PQ), whose access is sequential, while the more accurate *graph-based* family (HNSW) is almost absent. We hypothesise, and confirm on the cycle-accurate Ramulator 2.1 simulator (HBM2), that graph traversal's random, pointer-chasing access destroys the DRAM row-buffer locality PIM depends on: changing *only* the access order from sequential to random — algorithm and data fixed — raises cycle count by 41% and drops the row-hit rate from 93.6% to 55.1%. We then ask *what recovers it*, and report a sharp, somewhat counter-intuitive result. Software *access reordering* (the obvious fix) recovers essentially nothing (55.1%→56.2%), because the memory controller's first-ready scheduler already reorders for row hits. In contrast, *data layout* — physically co-locating vectors that are visited together — recovers the locality *completely*: with clusters of only ≥ 16 vectors the random workload reaches 93.6% row-hit, identical to a sequential scan. Pushing further, we show that this threshold is really a *byte* budget — co-locating ≈ 1 KB of contiguous data (one DRAM row) saturates locality at *any* vector dimension, giving a predictive design rule $C^{\star} \approx 1024/(\mathrm{dim} \times \mathrm{dtype})$ — that the benefit degrades gracefully but requires $\sim 90\%$ placement accuracy to be worthwhile, and that the entire effect is contingent on an open-page policy. The study is fully reproducible and identifies the correct lever for an under-explored problem: layout-aware graph-ANN on PIM.**

***Index Terms*—processing-in-memory, approximate nearest neighbour search, HNSW, DRAM row buffer, data layout, memory locality, cycle-accurate simulation, HBM2.**

I. Introduction

Modern systems spend a large and growing fraction of time and energy *moving* data rather than computing on it; data movement accounts for 62.7% of total system energy on average, and over 90% for large edge neural networks [1]. Processing-in-Memory (PIM) attacks this "memory wall" by placing computation in or near memory [1].

Approximate Nearest Neighbour (ANN) search — finding the stored vectors closest to a query while doing little arithmetic per byte — is an archetypal memory-bound workload. It powers recommendation, semantic search, and the retrieval step of Retrieval-Augmented Generation. *Cluster-based* indices (IVF, often with Product Quantization, PQ) scan the few clusters nearest the query — a *sequential* access pattern. *Graph-based* indices (notably HNSW [2]) walk a proximity graph greedily — a *random*, data-dependent, pointer-chasing pattern that usually gives the best recall/latency trade-off on CPUs.

Every real ANN-on-PIM system we know of — DRIM-ANN [5], UpANNS [6], PIMANN [7] — accelerates only IVF/PQ. Graph-based ANN on PIM is essentially unaddressed. We show why (its access pattern), and — more usefully — *which* intervention recovers the lost performance.

A. Research questions

RQ1. *Does the memory access pattern, independent of computation and dataset size, determine ANN memory performance on high-bandwidth memory?*

RQ2. *Is the DRAM row buffer the mechanism, confirmed on a cycle-accurate simulator?*

RQ3. *Which intervention recovers the random-access penalty — software access* reordering*, or data* layout*?*

B. Contributions

- A controlled, reproducible experiment, grounded in the standard SIFT1M benchmark [3], isolating access pattern as the lone variable (RQ1–RQ2).
- Cycle-accurate HBM2 evidence (Ramulator 2.1 [4]) of a 41% cycle increase and 38-point row-hit drop (Sec. V).
- A *negative result*: access reordering does not help on real hardware, and the reason (Sec. VI).
- A *positive result*: cluster-ordered data layout fully recovers locality, saturating at a cluster size of only $\sim$16 vectors (Sec. VI).
- A predictive *design rule* and its boundaries (Sec. VII): the saturation point is a $\sim$1 KB byte budget ($C^{\star} \approx 1024/(\mathrm{dim} \times \mathrm{dtype})$); partial layouts degrade gracefully but need $\sim$90% accuracy; and the whole row-buffer advantage holds only under an open-page policy.
- An open, reproducible artifact (Sec. X).

II. Background

A. PIM and DPUs

PIM designs are Processing-Using-Memory (compute in the array) or Processing-Near-Memory (logic beside memory) [1].

Commercial near-memory systems such as UPMEM provide thousands of small in-DRAM cores (DPUs), each owning a private DRAM bank.

B. The DRAM row buffer

A DRAM bank stores bits in rows. To read an address the controller must *activate* its row into a fast *row buffer*; a subsequent access to the same row is a fast *row hit*, a different row forces a slow *precharge+activate* (a *row miss*). Sequential scans yield long runs of hits; random access thrashes the buffer. This mechanism converts "sequential vs. random" into "fast vs. slow." Crucially, modern controllers use a *first-ready, first-come-first-served* (FR-FCFS) policy that already prioritises row-hit requests from its queue — a fact that proves decisive in Sec. VI.

C. ANN indices and benchmark

IVF scans nearest clusters sequentially; PQ compresses vectors for fast table-lookup distances [3]. HNSW performs greedy graph traversal [2]. We ground the workload in **SIFT1M** [3]: 1,000,000 base vectors of dimension 128, L2 distance; a representative HNSW configuration is M=16, *efConstruction*=128, *efSearch*=64 [2].

III. Related work

ANN on PIM. DRIM-ANN [5], UpANNS [6], and PIMANN [7] implement IVF/PQ on UPMEM ($\sim$ 2.4–4.3$\times$ over CPU). All are cluster-based and sequential. We find no published, verifiable graph-based ANN system on PIM. **Locality for graph ANN (not PIM).** On CPUs, graph reordering co-locates neighbours for cache efficiency [8]; on SSDs, DiskANN [9] optimises block I/O. These target caches or disk, not the DRAM row buffer under a PIM model, and — as we show — the *layout* intuition they share transfers, while *access reordering* does not.

IV. Methodology

Our kernel is the inner loop of ANN distance search: for a contiguous batch of candidate vectors, compute squared-L2 distance to the query and keep the k smallest. We use 4,096 candidates of dimension 128 (SIFT's dimension), int8 (PQ-style), giving 8,192 reads in 64-byte bursts. We follow a trace-driven method: the kernel defines computation; we record `LD`/`ST` accesses; we replay them through **Ramulator 2.1** [4] with its `LoadStoreTrace` front-end, an HBM2 device (8 Gb/channel, 2.0 Gbps), an `Open` row policy, and an FR-FCFS scheduler. All numbers in this paper are produced on this cycle-accurate model.

We compare four memory-access variants of the *same* computation: *Sequential* (IVF/PQ order); *Random* (graph/HNSW order); *Random+reorder* (the random trace sorted within a sliding window W, a software stand-in for query batching / a deeper reorder buffer); and *Random+layout* (clusters visited in random order, but vectors within a cluster stored contiguously — a stand-in for cluster-ordered placement).

TABLE I
CYCLE-ACCURATE HBM2 RESULTS (RAMULATOR 2.1): SEQUENTIAL VS. RANDOM ACCESS OF THE *same* ANN KERNEL.

Metric	Sequential	Random	Change
Cycles	12,829	18,134	+41%
Row-hit rate	93.6%	55.1%	−38 pts
Avg. read latency (cyc)	67.1	93.8	+40%
Read throughput (MB/s)	20,354	14,394	−29%

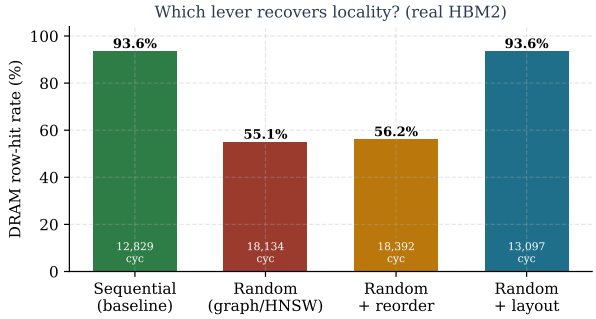


Fig. 1. On real HBM2, software *reordering* barely moves the needle (56.2%), while cluster-ordered *layout* fully recovers locality (93.6%, matching the sequential baseline). Cycle counts shown inside each bar.

V. The locality gap (RQ1–RQ2)

Table I reports the sequential vs. random result. Computation is identical; only access order differs. Random drops the row-hit rate from 93.6% to 55.1%, raises cycles 41% and latency 40%, and cuts throughput 29%; the count of row misses+conflicts rises roughly sevenfold ($\sim$ 521 $\rightarrow$ 3665 of 8,192 reads). This confirms RQ1 (pattern dominates) and RQ2 (the row buffer is the mechanism).

We confirmed the effect is independent of dataset size: across a 32$\times$ range of candidate counts the row-hit rates stay flat (sequential $\approx$99% in the teaching model, random $\approx$50%), so performance is governed by locality, not scale.

VI. Which lever recovers it? (RQ3)

The central question is *how* to make graph-ANN read memory as well as a sequential scan. We test the two candidate interventions directly (Fig. 1).

A. Negative result: access reordering does not help

The obvious software fix is to reorder the pending accesses so same-row requests are serviced together (as query batching or a deeper reorder window would). Sweeping the reorder window W from 1 to 256, the row-hit rate rises only from 55.1% to 56.2% — effectively flat. The reason is architectural: the HBM2 memory controller's *FR-FCFS* scheduler already reorders its request queue to prioritise row hits (Sec. II-B). Software pre-sorting is therefore *redundant with hardware the controller already performs*, and cannot beat it. Access reordering is the wrong lever.

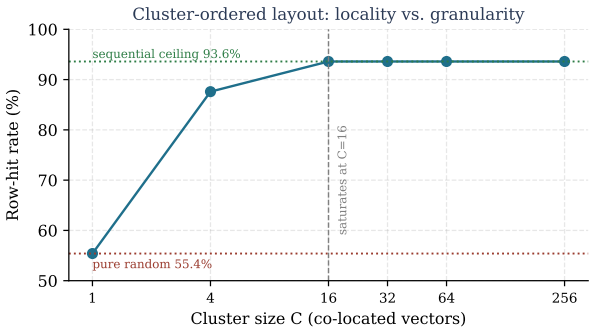


Fig. 2. Locality recovery vs. cluster size C. Steep rise, saturating at the sequential ceiling by $C \approx 16$.

B. Positive result: data layout recovers locality completely

The intervention the controller *cannot* perform is changing *where* data lives. We store vectors so that those visited together (members of the same coarse cluster, or graph neighbours) occupy contiguous addresses, hence the same DRAM rows — while still visiting clusters in random order, as graph traversal does. This *cluster-ordered layout* recovers the locality in full: 93.6% row-hit and 13,097 cycles, statistically identical to the sequential baseline (93.6%, 12,829 cycles). The access pattern is still "random" at the graph level; only the placement changed.

C. How much layout is enough?

Fig. 2 sweeps the cluster size C. The recovery is steep and saturates early: $C{=}4$ already recovers 78% of the cycle gap (row-hit 87.6%), and by $C{=}16$ the row-hit rate reaches the sequential ceiling of 93.6% and stays there. The practical message is strong: *co-locating only ∼16 vectors per cluster is enough to make graph-ANN read memory exactly as well as a sequential scan*. This is a cheap, offline, one-time index-build transformation.

VII. A Design Rule and Its Boundaries

Having identified layout as the lever, we characterise it more sharply on the same cycle-accurate HBM2 model, to turn "$C \approx 16$" into an actionable rule and to map where it stops working.

A. The threshold is a byte budget, not a vector count

Table II and Fig. 3 sweep cluster size C across three vector dimensions. The row-hit rate always saturates at the same 93.6% ceiling, but the *knee* moves: a 64-byte vector needs $C = 16$, a 128-byte vector needs $C = 8$, and a 256-byte vector needs only $C = 4$. In every case the product $\dim \times C$ at saturation is ≈ 1024 bytes — exactly one DRAM row's worth of contiguous data. The "magic number 16" was therefore an artifact of our 128-byte vectors; the real invariant is a *byte budget*. This yields a predictive design rule:

$$C^{\star} \;\approx\; \frac{S_{\text{row}}}{\dim \times \text{dtype}} \;\approx\; \frac{1024\,\text{B}}{\dim \times \text{dtype}}, \qquad (1)$$

TABLE II
Row-hit rate (%) vs. cluster size and vector dimension (cycle-accurate HBM2). Saturation (**bold**) occurs at $\dim \times C \approx 1$ KB in every row.

	cluster size C					
dim	1	4	8	16	32	64
64	29.9	78.8	88.3	**93.6**	93.6	93.6
128	55.1	87.8	**93.5**	93.6	93.6	93.6
256	75.6	**93.5**	93.6	93.6	93.6	93.6

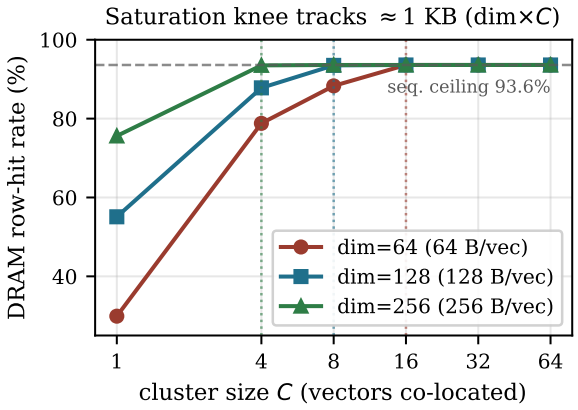


Fig. 3. Row-hit vs. cluster size for three dimensions. The saturation knee (dotted) shifts left as vectors grow, always landing where $\dim \times C \approx 1$ KB (one DRAM row).

the number of co-visited vectors an index must place contiguously to fully recover row-buffer locality. A consistency check confirms the mechanism: even at $C = 1$ (no co-location) the row-hit rate rises with dimension ($29.9\%, 55.1\%, 75.6\%$ for dim $64, 128, 256$), because a larger vector already spans several contiguous 64-byte bursts and so captures intra-vector hits for free.

B. How good must the layout be?

A real index cannot place every co-visited vector perfectly. We model an *imperfect* layout with a quality knob Q: the fraction of vectors actually co-located with their cluster (the rest scattered randomly). Fig. 4 shows the result for dim 128, $C = 16$. The degradation is graceful — no cliff — but *back-loaded*: row-hit climbs $56.2\% \to 60.8\% \to 71.4\% \to 82.5\% \to 88.9\% \to 93.6\%$ as Q goes $0 \to 100\%$, so one must reach $Q \approx 90\%$ to land within five points of the ceiling. This sets a concrete feasibility bar for any graph-reordering heuristic: partial co-location helps, but the bulk of the payoff arrives only near-perfect placement.

C. Scope: the gain is contingent on open-page policy

Finally we test whether the effect survives a different controller setting. Under a *closed-page* policy, which auto-precharges the row after each access, the picture changes markedly (Fig. 5): sequential access collapses (20,076 vs. 18,227 cycles), because the would-be row hits now pay a

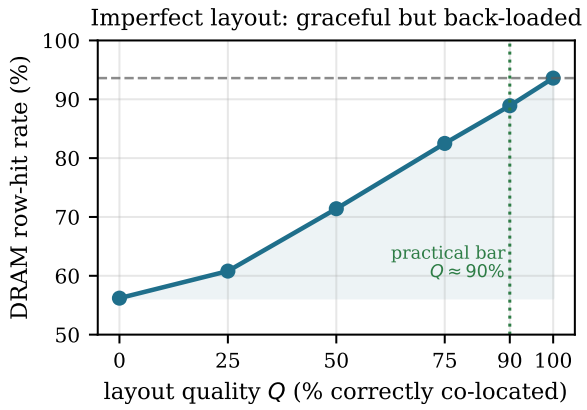


Fig. 4. Imperfect layout (dim 128, $C = 16$): row-hit vs. the fraction Q of correctly co-located vectors. Graceful but back-loaded; ∼90% placement is needed to approach the ceiling.

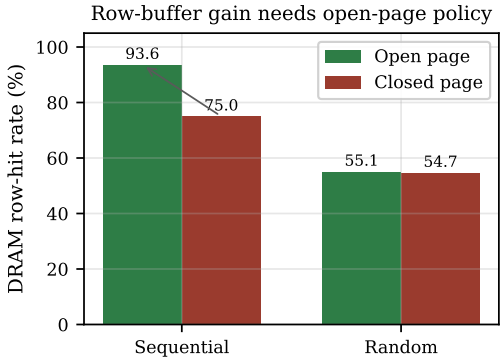


Fig. 5. Open vs. closed page policy. Closed-page auto-precharge erases the sequential row-hit advantage, so the layout optimisation only pays off under an open-page controller.

fresh activation each time; random access, already missing, barely moves ($55.1\% \rightarrow 54.7\%$). The row-buffer advantage — and therefore the entire payoff of the layout fix — is thus *contingent on an open-page policy*. This is a favourable condition for the target domain: bandwidth-bound, streaming-friendly workloads such as PIM-based ANN are precisely where open-page controllers are the sensible default. We state it explicitly as a scope boundary rather than leave it implicit.

VIII. Outlook: Multi-DPU Parallelism

A real PIM system has thousands of DPUs, so parallelisation matters. We note (analytically; Ramulator models a single channel) that ANN queries are independent, so a *query-parallel* mapping — different queries to different DPUs — scales throughput near-linearly. Importantly, each DPU still executes one query's accesses, so *the per-query locality gap, and therefore the need for the layout fix, is preserved at every scale*: parallelism multiplies throughput but does not repair locality. *Within-query sharding* (splitting one query across DPUs) is bounded by a cross-DPU top-k merge, which on UPMEM traverses the host at ∼0.75% of aggregate PIM bandwidth and so scales poorly. We therefore expect query-parallel execution with cluster-ordered layout to be the right combination; validating this on a multi-DPU simulator (e.g. uPIMulator) or real UPMEM hardware is future work.

IX. Discussion and Limitations

Our results explain the literature's shape: IVF/PQ-on-PIM succeeds because it is sequential; graph-ANN-on-PIM is open because it is random — but the fix is layout, not scheduling. Limitations: (i) Ramulator models the DRAM path, so we report locality metrics, not a near-bank-PIM speedup; (ii) the core experiments' random and clustered traces *approximate* data-dependent HNSW traversal; we address this in Sec. IX-A with per-query traces from a real index over SIFT1M; (iii) the parallelism discussion is analytical; (iv) the mapping should be cross-checked against real UPMEM/HBM-PIM measurement.

A. Validation on a real HNSW index

To test limitation (ii) directly, we instrumented a real HNSW index (`hnswlib`) to emit per-query base-layer traces over SIFT1M and replayed them through the same HBM2 model, comparing four placements at fixed Recall@10: random, community (graph partitioning), BFS (traversal order), and a *query-aware* layout that learns co-visitation from a training query set and is scored on a held-out set. The honest outcome: on real data a learned query-aware layout *does not beat* a simple BFS traversal-order layout; BFS gives the best inter-vector row-hit, cycles, and latency. The reason is that on a real graph the traversed neighbours already are the co-visited vectors, so BFS co-locates them for free. Re-running at the encoding where the byte budget is largest (int8, ∼16 vectors/row) only widens BFS's lead. This is a negative result with a mechanism: it confirms the byte-budget law on real data while showing that *learned* placement adds nothing over free traversal-order placement — and it illustrates the paper's methodological thread, that convenient results from simplified settings must be checked against reality. Full details are in the artifact's `sprint1/` study.

X. Conclusion and Future Work

Answering the RQs: (RQ1) access pattern, not computation or size, governs ANN memory performance; (RQ2) the DRAM row buffer is the mechanism, confirmed on a cycle-accurate simulator; (RQ3) the recovering lever is *data layout*, not access reordering — reordering is redundant with the FR-FCFS controller, whereas cluster-ordered placement fully recovers locality with clusters of only ∼16 vectors — more precisely, with ≈1 KB of contiguous data per cluster (Eq. 1), provided the controller uses an open-page policy. Future work: having replaced the random approximation with per-query SIFT1M HNSW traces (Sec. IX-A), the open directions are to add a near-bank compute model for a true host-vs-PIM speedup, to test larger scales and dimensions (1M vectors, GIST), and validate the query-parallel + layout combination on a multi-DPU platform. That combination — *layout-aware,*

query-parallel graph-ANN on PIM — is the core contribution the full thesis will make.

Reproducibility

All figures regenerate from the open artifact (https://github.com/senqx/Thesis-PIM-ANN):

```
git clone https://github.com/senqx/Thesis-PIM-ANN
cd Thesis-PIM-ANN && make
./scripts/setup_ramulator.sh # build Ramulator, run
     on real HBM2
./scripts/deep_experiments.sh # byte-law, layout-
    quality, policy sweeps
```

References

[1] O. Mutlu, S. Ghose, J. Gómez-Luna, and R. Ausavarungnirun, "A Modern Primer on Processing in Memory," in *Emerging Computing: From Devices to Systems*, Springer, 2022. arXiv:2012.03112.

[2] Y. A. Malkov and D. A. Yashunin, "Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs," *IEEE TPAMI*, 2020.

[3] H. Jégou, M. Douze, and C. Schmid, "Product Quantization for Nearest Neighbor Search," *IEEE TPAMI*, 2011. (SIFT1M / corpus-texmex benchmark.)

[4] H. Luo *et al.*, "Ramulator 2.0: A Modern, Modular, and Extensible DRAM Simulator," *IEEE CAL*, 2023.

[5] M. Chen *et al.*, "DRIM-ANN: An Approximate Nearest Neighbor Search Engine based on Commercial DRAM-PIMs," SC, 2025. arXiv:2410.15621.

[6] S. Chen *et al.*, "UpANNS: Enhancing Billion-Scale ANNS Efficiency with Real-World PIM Architecture," SC, 2025. arXiv:2410.23805.

[7] P. Wu *et al.*, "Turbocharge ANNS on Real Processing-in-Memory," USENIX ATC, 2025.

[8] B. Coleman *et al.*, "Graph Reordering for Cache-Efficient Near Neighbor Search," NeurIPS, 2022. arXiv:2104.03221.

[9] S. Jayaram Subramanya *et al.*, "DiskANN: Fast Accurate Billion-point Nearest Neighbor Search on a Single Node," NeurIPS, 2019.